Les classes, les objets et les méthodes

4) Refaites les exercices de la leçon sur les objets et les attributs mais, cette fois-ci, en utilisant des méthodes d'instances plutôt que des fonctions statiques. Considérez les définitions suivantes à compléter (dans deux fichiers Date.java et Personne.java) :

```
public class Date {
  public int jour, mois, annee;
}

public class Personne {
  public String nom, prenom;
  public Date ddn; // date de naissance
}
```

Écrivez un programme qui demande à l'utilisateur de faire la saisie au clavier de deux *Personne*s, les réaffiche à l'écran, et détermine laquelle est la plus âgée et laquelle précède l'autre dans le bottin.
Il est important de bien modulariser votre solution : utilisez des méthodes pour chaque opération identifiable. Ecrivez donc, au minimum, les méthodes suivantes :

a) `public void saisie()` (dans la classe *Date*) qui complète la *Date* sur laquelle la méthode est appelée en demandant à l'utilisateur le jour, le mois et l'année. On supposera, sans le vérifier, que la date rentrée est valide.

b) `public void saisie()` (dans la classe *Personne*) qui complète la *Personne* sur laquelle la méthode est appelée en demandant à l'utilisateur son nom, son prénom et sa date de naissance. On supposera que le nom et le prénom d'une personne ne contiennent qu'un seul mot.

c) `public void affiche()` (dans la classe *Date*) qui affiche la *Date* sur laquelle la méthode est appelée.

d) `public void affiche()` (dans la classe *Personne*) qui affiche le nom, le prénom et la date de naissance de la *Personne* sur laquelle la méthode est appelée.

e) `public int compareTo(Date other)` (dans la classe *Date*) qui compare la date sur laquelle la méthode est appelée avec la date *other*. Celle-ci renverra un entier positif (quelconque) si *this* est plus « grand » que *other*, négatif si *this* est plus « petit » que *other* et 0 si *this* et *other* sont identiques. Que signifie qu'une `Date` est plus grande qu'une autre ?

f) `public int compareToAge(Personne other)` (dans la classe *Personne*) qui compare la *Personne* sur laquelle la méthode est appelée avec la personne *other* selon leur âge (en jours). Elle renverra un positif si *this* est plus agé que *other*, un négatif si *other* est plus agé que *this* et 0 si les deux *Personnes* sont nées le même jour.

g) `public int compareToNomPrenom(Personne other)` (dans la classe *Personne*) qui compare la personne sur laquelle la méthode est appelée avec la personne `other` selon l'ordre alphabétique de leur nom et de leur prénom.

2) Complétez votre programme afin qu'il demande à l'utilisateur d'entrer une série de ***n*** *Personne*s (la valeur de *n* est indiquée au préalable par l'utilisateur), les range dans un tableau et les affiche :

```
public static void exercice2() {
   Array<Personne> tab = new Array<>();
   saisie(tab);
   affiche(tab);
}
```

3) Complétez votre programme pour afficher la personne la plus jeune du tableau. Utilisez pour cela votre méthode d'instance compareToAge :

```
public static void exercice3() {
   Array<Personne> tab = new Array<>();
   saisie(tab);

   System.out.println("La personne la plus jeune est : ");
   Personne p = trouveLaPlusJeune(tab);
   p.affiche();
}
```

4) Ajoutez un constructeur de *Date* qui reçoit trois entiers et construit une *Date* sur base de ces trois valeurs. Essayez ensuite de créer quelques *Date*s à l'aide de ce constructeur et affichez-les ensuite pour montrer qu'il fonctionne correctement. Remarquez que le constructeur sans argument n'existe plus. Définissez-le.

   Ajoutez un constructeur de *Personne* qui reçoit deux *String*s et trois entiers. Celui-ci doit construire une *Personne* et sa *Date* de naissance sur base des valeurs reçues. Utilisez le constructeur de *Date* dans le constructeur de *Personne*. Essayez ensuite de créer quelques *Personne*s à l'aide de ce constructeur et affichez-les ensuite pour montrer qu'il fonctionne correctement. De même, remarquez que le constructeur sans argument n'existe plus. Définissez-le.

5) Ajoutez une méthode d'instance public String toString() à vos classes *Date* et *Personne*. Utilisez-les dans les méthodes affiche() correspondantes. Créez quelques objets et affichez les ensuite à l'aide de println() et à l'aide de la méthode affiche() pour montrer leur fonctionnement.

6) Ajoutez une méthode d'instance public int getAge() à la classe *Personne*. Celle-ci doit renvoyer l'âge (en années) de la *Personne* en question. Pour cela, vous aurez besoin de connaître la date du jour. Basez-vous sur le code suivant :

```
LocalDate now = LocalDate.now(); // (import java.time.LocalDate;)
int jour     = now.getDayOfMonth();
int mois     = now.getMonthValue();
int annee    = now.getYear();
```

   Utilisez cette méthode pour afficher l'âge de plusieurs *Personne*s bien choisies.

Exercices récapitulatifs sur les strings, les vectors et les structs.

(voir document annexe)